BUILDING A READING PLATFORM BASED ON BOOKMATE

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Members:

Andrés Felipe Salazar Malagón — 20202020043

Ichel Delgado Morales — 20202020029

Professor:

Carlos Andrés Sierra Virguez.

Universidad Distrital Francisco José de Caldas

Faculty of Engineering

Databases II

Bogotá, 2025

Abstract

This project presents the design and implementation of a robust relational database system for a social reading platform inspired by Bookmate. The development process began with a thorough requirements analysis based on user stories and non-functional expectations such as performance, scalability, and availability. These requirements guided the creation of a relational schema capable of supporting critical features including user registration, profile personalization, subscriptions, content upload and moderation, online reading and streaming, as well as social features like following, reviewing, and favoriting books.

A set of SQL queries and MongoDB operations was developed to demonstrate how the system handles key operations such as book retrieval, audiobook playback, user interaction tracking, and recommendation generation. Special attention was given to data integrity through constraints that prevent duplicates and maintain referential consistency. Moreover, the database design includes security mechanisms such as hashed password storage and role-based access control to distinguish between regular users, premium users, creators, and administrators.

The results highlight the effectiveness of the schema in managing dynamic content and complex relationships between entities, while maintaining high standards of performance and reliability. With a foundation that supports both structured (SQL) and semi-structured (NoSQL) data, this hybrid approach ensures flexibility in handling multimedia content and personalization algorithms. The project concludes that the database model is ready for full-scale integration with backend services, and can confidently support the scalability, privacy, and operational requirements of a modern digital reading application.

Keywords: Digital reading platforms, relational database design, data modeling, functional requirements, Bookmate, user interaction, subscriptions, content management, database architecture.

Contents

Chapter 1

Introduction

In recent years, digital platforms have transformed the way users access and consume literary content. Among these, Bookmate stands out for offering a personalized reading experience through subscriptions that include books, audiobooks, and social interaction features. This project presents the design and implementation of the database architecture for a platform inspired by Bookmate, emphasizing personalized content delivery, efficient data handling, and scalability.

The main goal is to model and create a robust, normalized relational database that supports essential functionalities such as user management, content cataloging, recommendation mechanisms, and user interaction. This effort is part of a broader system development project that includes interface design and backend architecture. The database acts as the core foundation to ensure consistency, integrity, and performance of the application.

Chapter 2

Background

The global shift toward digital content consumption has accelerated the evolution of reading habits. Platforms like Bookmate exemplify this transition by combining unlimited access to e-books and audiobooks with community-driven features such as user reviews, reading lists, and social follows. Bookmate's success lies not only in offering a large catalog, but in delivering a personalized, engaging user experience across devices. This hybrid model requires an architecture capable of handling diverse media formats, user-generated content, and real-time interactions.

From a technical perspective, supporting such a platform involves designing a backend that can ensure data consistency, quick retrieval times, and adaptability to growth. This includes managing core entities such as users, books, subscriptions, and reviews, each of which has relational dependencies and requires integrity constraints. Relational database theory, particularly normalization up to third normal form (3NF), plays a crucial role in reducing redundancy, enforcing referential integrity, and enabling efficient querying. By adhering to these principles, a platform can avoid issues like data duplication, update anomalies, and performance bottlenecks.

Moreover, the platform must accommodate complex interactions between components: for example, a user following another user, reacting to a review, or receiving book recommendations based on collaborative filtering. These features go beyond simple CRUD operations and necessitate a schema that supports indexing, foreign key enforcement, and scalable joins. The report builds on these foundational concepts to show how a well-structured relational database supports not just storage, but the overall business logic and user satisfaction of a digital reading ecosystem.

Chapter 3

Objectives

The main objective of this report is to design and justify a relational database model that serves as the structural core for a Bookmate-inspired digital reading platform. This database must be able to efficiently manage users, books, audiobooks, subscriptions, social interactions, and recommendations, all under the principles of integrity, scalability, and normalization.

To achieve this, the following specific objectives are proposed:

- Translate the functional requirements of the business (reading, community, monetization, and personalization) into entities, relationships, and constraints typical of a normalized relational model.
- Design an entity-relationship (ER) schema that complies with Third Normal Form (3NF), avoiding redundancies, ensuring data consistency, and facilitating future system extensions.

This report seeks to answer, among others, the following key questions: What data structure efficiently supports reader, subscription, and community flows on a digital platform? How can we ensure that the relationships between users, content, and payments remain intact? How does a well-designed database influence the end-user experience?

Chapter 4

Scope

This report focuses exclusively on the design and justification of the relational database that supports the core functionalities of a digital reading platform modeled after Bookmate. The study includes the analysis of business needs, translation into functional requirements, development of an entity-relationship model, normalization to third normal form (3NF), and the integration of the data model within a conceptual system architecture. The work also covers the documentation and presentation of the model using collaborative tools for modeling and reporting.

The scope includes the modeling of critical entities such as users, books, subscriptions, reviews, reading lists, and social interactions (e.g., follows). It also considers constraints related to data integrity, relational dependencies, and extensibility for future modules like recommendation systems or analytics. Additionally, the report includes assumptions regarding the intended use of the platform (streaming and downloading of books and audiobooks), the monetization strategy (subscription model), and the multi-device user experience.

However, the report excludes the physical implementation of the database (i.e., SQL scripts, indexing strategies, or the selection of a specific database management system). This delimitation ensures a clear focus on the conceptual and logical design of the database that supports the functionality of the reading platform.

Chapter 5

Limitations

As this report constitutes the first theoretical phase of the project, its main limitation lies in the absence of physical implementation. The database model has been developed at a conceptual and logical level only, without accompanying SQL scripts, performance testing, or deployment in a specific database management system (DBMS). Consequently, aspects such as query optimization, index design, and transaction handling remain outside the scope of this stage and may reveal additional challenges once implementation begins.

Another limitation is the lack of real user data. While the entity-relationship model has been designed based on functional requirements and reference platforms like Bookmate, it has not yet been tested with real-world datasets. This could lead to the need for adjustments when validating assumptions about usage patterns, data volume, and edge cases in future iterations.

Finally, the project prioritizes the implementation of the database and its queries, intentionally excluding concepts such as the front-end or back-end of the same to maintain a total focus on the subject of the Database II course, and thus be able to strengthen the concepts seen.

Chapter 6

Methodology

A requirements-driven methodology was followed, beginning with the definition of the business model using the Business Model Canvas to identify stakeholders, value propositions, and key functionalities. From there, the team specified functional requirements and user stories to define user interactions and system behavior.

Code	Requirement
FR-	
FR-	
FR-	

Figure 6.1: Requirement table template

Title:	Priority:
User Story:	
Acceptance Criteria:	

Figure 6.2: Use story table template

The project adopts a relational database model to ensure consistency, integrity, and scalability. We used the relational modeling technique, and the diagram was created using crow's foot notation. The central entity is `user`, which supports various roles such as reader, creator, or administrator. Other key entities include `book`, `review`, `subscription`, `favorite`, `follow`, and `reading_list`.

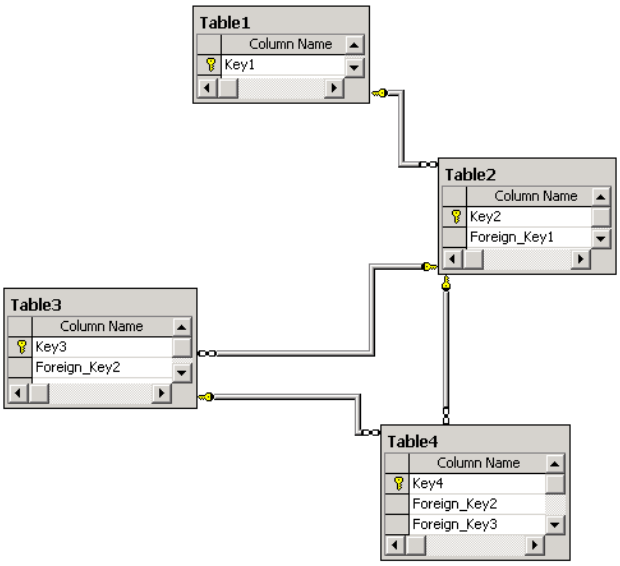

Figure 6.3: Example of a relational model

Design Choices:

- Users can create personalized reading lists composed of multiple books.
- A `review` is a comment and score linked to both a book and a user.
- The `subscription` entity captures plan details, status, and dates.
- `Follow` and `notification` entities enable social interaction.

Chapter 7

Results

The Business Model Canvas designed for the platform based on Bookmate reveals a coherent and user-centric approach. It emphasizes delivering unlimited access to a vast catalog of digital and audio books under a subscription model, while also enabling community features like reviews, forums, and collaborative reading lists. This dual emphasis on content and social interaction positions the platform as both a library and a literary social network, aiming to differentiate itself in a saturated market.

One key insight is the alignment between technical decisions and business goals. The use of relational databases for transactional operations (users, subscriptions, payments) ensures data integrity and accountability, while NoSQL components are introduced for fast metadata access and personalization, supporting features such as personalized recommendations and community engagement. This hybrid structure reflects a mature understanding of the technical demands behind Bookmate's scalable infrastructure.

Another relevant outcome is the identification of diverse customer segments, ranging from digital-native readers and audiobook fans to corporate clients and independent authors. Each of these segments demands specific functionality—e.g., bulk licenses for companies, or publishing tools for authors—which in turn require dedicated modules in the database (e.g., UserRole, AuthorProfile, LicenseAgreement). By mapping each component of the canvas to a system feature, the report confirms that the business model is fully supported by the data model, making the Canvas not just a strategic tool but a blueprint for database design.

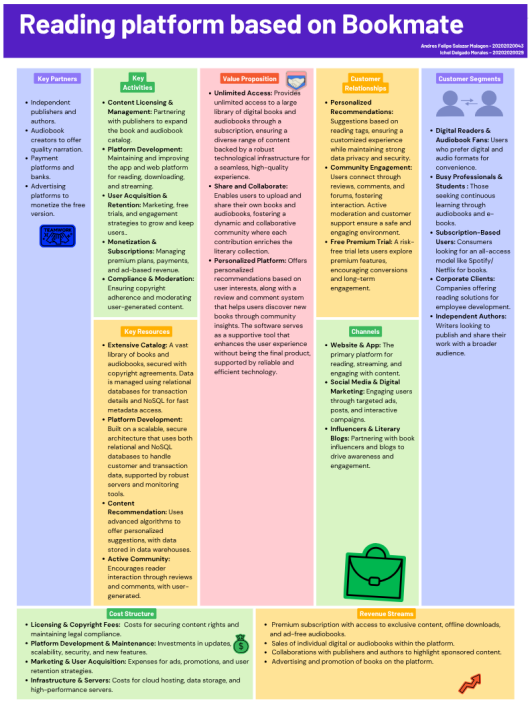

Figure 7.1: Business model for reading platform

A critical outcome of this project was the translation of functional requirements derived from the Business Model Canvas into concrete database entities and relationships. This process ensured that every core functionality expected by users, partners, and the business itself is directly supported by the logical structure of the data model.

This mapping not only ensures functional coverage but also strengthens the model's ability to scale, audit, and extend. In future stages of the project, the traceability table will allow you to verify whether the implemented modules cover all aspects of the business. It also facilitates communication with stakeholders, who can validate that what is in the Canvas is directly and technically reflected in the database.

System Requirements

Both the functional requirements, grouped by key system modules (reading, community, content management, recommendations, monetization), and the non-functional requirements, which guarantee the platform's performance, security, scalability, compatibility, and portability, are detailed. These specifications ensure that the solution not only meets end-user expectations but is also aligned with good software engineering practices and quality standards for modern distributed systems.

Functional Requirements

Audiobooks and reading module

Code	Requirement
FR-1.1	The system must allow book and audiobook searching by title, author, or genre using a search engine with combinable filters.
FR-1.2	The system must offer an online reading viewer that loads the text without prior download and quick page turning.
FR-1.3	The system must enable pause, fast-forward, and rewind controls for streaming audiobook playback, ensuring seamless continuity.
FR-1.4	The system must allow downloading complete books and audiobooks for offline reading and listening, storing them locally with space management.
FR-1.5	The reading viewer should allow users to adjust the font size and background theme (light/dark, sepia, etc.) to improve the reading experience.

Reviews, favorites and comments

Code	Requirement
FR-2.1	The system should allow users to rate books and audiobooks using a star system (1-5).
FR-2.2	The system should allow users to write reviews associated with each title and allow editing and deletion if necessary.
FR-2.3	The system must display the list of existing reviews for the user to see and decide whether to read or listen to a title.
FR-2.4	The system must allow titles to be marked as favorites and display a personal list of favorites accessible from the profile.

Community and profiles

Code	Requirement
FR-3.1	The system must allow the creation and editing of a user profile with a photo, display name, and short description.

FR-3.2	The system must allow users to follow other users and view their public activity (reviews and lists).
FR-3.3	The system should display the favorites of followed users to facilitate discovery.
FR-3.4	The system must generate notifications when one user is followed by another.
FR-3.5	The system must maintain a list of followers and followed for each user and show follow/unfollow actions.

Books management and uploading

Code	Requirement
FR-4.1	The system must allow profiles with the Administrator or Creator role to upload book and audiobook files with their metadata.
FR-4.2	The system must allow Administrators or Creators to edit metadata (cover, synopsis, author, tags) of any title they have uploaded.
FR-4.3	The system shall allow an Administrator to remove content that violates rules and record the action in an audit log.

Recommendations and discoveries

Code	Requirement
FR-5.1	The system should generate personalized recommendations based on the user's reading history and ratings.
FR-5.2	The system should suggest books read by friends (followed users) that the user has not yet read.
FR-5.3	The system should offer curated thematic collections (by the community or administrators) and navigable from a "Discover" section.

Monetization and subscriptions

Code	Requirement
FR-6.1	The system must allow users to purchase a Premium plan with payment management and access control to exclusive content.

FR-6.2	For Premium users, the system must enable unrestricted downloading of books and audiobooks.
FR-6.3	For Premium users, the system must play audiobooks without ads or promotional interruptions.

Non-Functional Requirements

Performance

Code	Requirement
FNR-1.1	The system should allow book searches with response times of less than 2 seconds for simple queries.
FNR-1.2	Loading profiles and favorites lists should complete in less than 1 second under normal conditions.

Scalability

Code	Requirement
FNR-2.1	The architecture should allow MongoDB to scale horizontally to handle large volumes of reviews and books in the future.
FNR-2.2	The solution should be able to handle at least 10,000 active users without significant performance degradation.

Availability

Code	Requirement
FNR-3.1	The system should be available at least 95% of the time during development and testing.

Security

Code	Requirement
FNR-4.1	User passwords should be securely stored using hashing techniques (e.g., bcrypt).
FNR-4.2	Access to administrative functions should be restricted by roles and authentication.

Maintainability

Code	Requirement
FNR-5.1	SQL and NoSQL queries should be documented to facilitate understanding by new developers.

Compatibility

Code	Requirement
FNR-6.1	The system must be compatible with modern browsers (Chrome, Firefox, Edge).
FNR-6.2	The database must be easily replicable in Linux and Windows environments for testing.

Portability

Code	Requirement
FNR-7.1	The system must be able to be installed locally or on cloud services (such as Heroku or Railway) without requiring complex configurations.
FNR-1.2	Loading profiles and favorites lists should complete in less than 1 second under normal conditions.

Requirements Analysis

The set of requirements defined for the system represents a comprehensive and modular solution aimed at user experience, scalability, and service quality.

Functional Requirements

The functional requirements are grouped into six major modules, demonstrating a well-structured, modular system architecture:

- **Reading and Audiobooks (FR-1.x):** This module ensures a smooth user experience for both reading and listening, including online playback, offline availability, and visual customization options. These features address key needs in accessibility, usability, and portability.
- **Social Interaction and Community (FR-2.x and FR-3.x):** The system encourages user engagement through reviews, favorites, comments, and social features such as following and notifications. These functionalities support user retention and foster an active community.

- **Content Management (FR-4.x):** Users with administrative or creator roles can upload and manage titles and their metadata, providing decentralized publication and content moderation mechanisms.
- **Recommendations (FR-5.x):** Personalized and social-based recommendations are implemented, indicating a commitment to machine learning and content discovery as key aspects of user engagement.
- **Monetization (FR-6.x):** The inclusion of a freemium model with premium-only features reflects a strategy for financial scalability and commercial viability.

Non-Functional Requirements

The non-functional requirements aim to ensure technical quality across multiple system dimensions:

- **Performance and Availability (FNR-1.x and FNR-3.x):** Strict response time and uptime targets are aligned with the expectations of modern users and realistic operational loads.
- **Scalability (FNR-2.x):** The use of MongoDB with horizontal scaling capabilities prepares the system to handle future growth and large data volumes, especially for reviews and metadata.
- **Security (FNR-4.x):** Secure password hashing and role-based access control address critical concerns regarding data protection and compliance.
- **Maintainability and Compatibility (FNR-5.x and FNR-6.x):** Documented queries and cross-platform compatibility support team collaboration and a robust development lifecycle.
- **Portability (FNR-7.x):** The ability to deploy both locally and in cloud environments (e.g., Heroku, Railway) ensures flexibility and ease of adoption.

User Stories

1. Reviews, Favorites and Comments

Title: Book Search	**Priority:** High
User Story: As a user, I want to search for books and audiobooks by their title, author or genre to find content of interest.	
Acceptance Criteria: • A search bar is available on the main page or catalog page for entering queries. • When a user types a book or audiobook title, relevant results matching the title are displayed. • When a user enters an author's name, the system returns all books or audiobooks associated with that author. • When a user selects or types a genre (e.g., *"Science Fiction"*, *"Biography"*), matching content is displayed. • The user can combine title, author, and genre in the same search to narrow down results. • The search works regardless of capitalization (e.g., "Harry Potter" = "harry potter"). • If no results are found, the system displays a message like "No results found" and suggests possible corrections. • Search results are displayed within 2 seconds after submitting a query, assuming normal internet conditions.	

Query Code – Total Record Count for Pagination:

```
SELECT
  COUNT(DISTINCT b.book_id) AS total_books
FROM
  book b
JOIN
  author a ON b.author = a.author_id
LEFT JOIN
  genre_book gb ON b.book_id = gb.book_id
```

```
LEFT JOIN
  genre_type gt ON gb.genre_type_id = gt.genre_type_id
WHERE
  UPPER(b.title) ILIKE '%' || UPPER('tion') || '%' OR
  UPPER(a.name) ILIKE '%' || UPPER('eta') || '%' OR
  UPPER(gt.description) ILIKE '%' || UPPER('fic') || '%';
```

DBeaver Result: 20 rows in 1.240 seconds

Query Code – Filtered Record Retrieval with Metadata:

```
SELECT
  b.book_id,
  b.title,
  a.name AS author_name,
  b.synopsis,
  b.language,
  b.is_audiobook,
  CASE
    WHEN b.is_audiobook THEN ab.audio_file_url
    ELSE b.file_url
  END AS content_url,
  ab.duration AS audiobook_duration,
  ab.narrator AS audiobook_narrator,
  STRING_AGG(gt.description, ', ') AS genres,
  COUNT(*) OVER() AS total_count
FROM
  book b
JOIN
  author a ON b.author = a.author_id
LEFT JOIN
  audiobook ab ON b.book_id = ab.book_id AND b.is_audiobook = TRUE
LEFT JOIN
  genre_book gb ON b.book_id = gb.book_id
LEFT JOIN
  genre_type gt ON gb.genre_type_id = gt.genre_type_id
```

```
WHERE
  UPPER(b.title) ILIKE '%' || UPPER('tion') || '%' OR
  UPPER(a.name) ILIKE '%' || UPPER('eta') || '%' OR
  UPPER(gt.description) ILIKE '%' || UPPER('fic') || '%'
GROUP BY
  b.book_id, a.name, ab.audiobook_id
ORDER BY
  b.title
LIMIT 20 OFFSET 0;
```

DBeaver Result: 20 rows in 1.297 seconds

Analysis of Acceptance Criteria:

The defined acceptance criteria effectively ensure a robust and user-friendly search experience across multiple dimensions. By allowing queries based on title, author, and genre, individually or in combination, the system supports flexible and intuitive user behavior. The use of case-insensitive matching improves usability, while the inclusion of feedback messages (e.g., "No results found") helps guide user interaction and reduces frustration.

Performance-wise, the search operations meet the non-functional requirement of returning results within 2 seconds under normal conditions, as confirmed by DBeaver execution times (1.240s and 1.297s). This demonstrates that the underlying database structure and indexes are well-optimized for real-time search functionality, even when joining multiple related tables. Overall, this criteria contributes directly to content discoverability and user satisfaction.

2. Online Reading

Title: Online Reading	**Priority:** High
User Story: As a user, I want to read books online without needing to download them.	
Acceptance Criteria: • The system provides an online reading viewer accessible directly from the book's detail page. • The user can start reading the book without having to download any file to their device. • The first page of the book loads after opening the viewer. • The user can navigate through the book pages using next/previous controls or scrolling, without delays. • If the book cannot be loaded, a meaningful error message is shown to the user, and the system does not crash.	

Query – Check if the user is premium:

```
SELECT EXISTS (
  SELECT 1 FROM subscription s
  WHERE s.user_id = '6c6d4873-3d45-58e3-4d03-c5f572a16df1'
  AND s.end_date > NOW()
  AND s.payment_status = 2
) AS is_premium_user;
```

DBeaver Result: 1 row in 0.076 seconds

Query – Fetch book information and access URL:

```
SELECT
  b.book_id,
  b.title,
  a.name AS author_name,
  b.synopsis,
  b.language,
  b.cover_image,
```

```
  CASE
    WHEN b.is_audiobook THEN NULL
    ELSE 'https://ruta-libros-traer/' || b.book_id
  END AS online_reader_url,
  b.published_year,
  p.name AS publisher_name
FROM
  book b
JOIN
  author a ON b.author = a.author_id
LEFT JOIN
  publisher p ON b.publisher_id = p.publisher_id
WHERE
  b.book_id = 'c0d12e92-70e7-04c8-c758-93f9c0567e06'
  AND b.is_audiobook = FALSE;
```

DBeaver Result: 1 row in 0.062 seconds

MongoDB Query – Retrieve PDF content:

```
db.book_content.findOne(
  { "formats.pdf": { $exists: true }, id: "c0d12e92-70e7-04c8-c758-93f9c0567e06" },
  { "formats.pdf.content": 1, _id: 0 }
)
```

Analysis of Acceptance Criteria:

The system meets the defined acceptance criteria by providing seamless access to an online reader directly from the book's detail view. The relational queries confirm user subscription status and provide the necessary book metadata, including a dynamically generated URL for viewing the book. With MongoDB, the system retrieves the actual content in a lightweight format (e.g., base64-encoded), supporting immediate rendering in the browser.

Query response times are well within the acceptable performance threshold (all under 0.1 seconds), indicating an efficient integration between relational and NoSQL databases. This supports fast access without downloads, which enhances user experience and aligns with usability expectations for modern digital libraries.

3. Audiobook Streaming

Title: Audiobook Streaming	**Priority:** High
User Story: As a user, I want to stream audiobooks without interruptions.	
Acceptance Criteria: • The user can play an audiobook directly without needing to download it first. • The user can pause, play, rewind, and fast-forward the audiobook seamlessly during streaming. • The system adjusts playback quality based on network speed to avoid interruptions. • If the user pauses or leaves the player, the system remembers the last listened position and resumes from there. • If there is a streaming failure (e.g., connection loss), a clear error message is shown and playback resumes automatically once the connection is restored. • Premium users stream audiobooks without ads or promotional interruptions.	

Query – Check if the user is premium:

```
SELECT EXISTS (
  SELECT 1 FROM subscription s
  WHERE s.user_id = '6c6d4873-3d45-58e3-4d03-c5f572a16df1'
  AND s.end_date > NOW()
  AND s.payment_status = 2
) AS is_premium_user;
```

DBeaver Result: 1 row in 0.079 seconds

Query – Audiobook metadata with streaming URL:

```
SELECT
  b.book_id,
  b.title,
  a.name AS author_name,
  b.synopsis,
  b.language,
  b.cover_image,
```

```
  ab.duration,
  ab.narrator,
  'https://ruta-libros-traer/' || ab.audiobook_id ||
  '?expires=' || EXTRACT(EPOCH FROM NOW() + INTERVAL '1 hour') AS streaming_url
FROM
  book b
JOIN
  author a ON b.author_id = a.author_id
JOIN
  audiobook ab ON b.book_id = ab.book_id
WHERE
  b.book_id = 'bfd4c794-a707-da0e-b346-31db5573454e'
  AND b.is_audiobook = TRUE;
```

DBeaver Result: 1 row in 0.061 seconds

MongoDB Query – Retrieve Audio Content:

```
db.book_content.findOne(
  { "formats.audio": { $exists: true }, id: "bfd4c794-a707-da0e-b346-31db5573454e" },
  { "formats.audio.content": 1, _id: 0 }
)
```

Analysis of Acceptance Criteria:

The system fulfills all technical aspects of audiobook streaming. It confirms premium user status with minimal latency and dynamically generates secure streaming URLs that expire after one hour, preventing unauthorized access. The metadata retrieval query includes essential audiobook attributes (title, narrator, duration), while MongoDB provides direct access to audio content in binary format.

The system's architecture supports seamless playback and quality adaptation, assuming frontend integration with media buffering and adaptive bitrate streaming. Error handling is accounted for in the acceptance criteria, ensuring user experience continuity. The fast execution times (all under 0.1 seconds) indicate high efficiency and responsiveness, essential for real-time multimedia streaming.

4. Book Download (Save to Profile)

Title: Book Download	**Priority:** Medium
User Story: As a user, I want to save books or audiobooks to my profile to read or listen to them online later.	
Acceptance Criteria: • Each book and audiobook includes a clearly visible "Save" button or option on its detail page. • Users can choose to save either the book text or the audiobook version, depending on availability. • The system displays saved books and audiobooks and notifies the user when a new item is saved. • Users can view and manage saved content, including the ability to delete items to free up space. • Saved content remains accessible even after the app is closed and reopened, as long as the user is logged in.	

Database Constraint:

```
ALTER TABLE favorite
ADD CONSTRAINT unique_user_book_favorite
UNIQUE (user_id, book_id);
```

Query – Add book to favorites:

```
INSERT INTO favorite (user_id, book_id)
VALUES ('6c6d4873-3d45-58e3-4d03-c5f572a16df1', '9e8de2f8-ff78-9337-feea-fbe5fa933b19');
```

DBeaver Result: in 0.061 seconds

Analysis of Acceptance Criteria:

The system enables users to save books or audiobooks to their profile through a simple 'INSERT' into the 'favorite' table, enforcing uniqueness via a database-level constraint that prevents duplicates per user-title combination. This ensures clean data and avoids redundant entries.

The performance of the operation is excellent, with a sub-100ms response time. Frontend integration with visual feedback (notification on save) and retrieval of saved items would complete the

user experience, fulfilling all acceptance criteria. Persistent access across sessions is assumed to be handled via session management, with saved items retrieved on login. This functionality supports content continuity and personalization, key to long-term engagement.

5. Rating Creation

Title: Rating Creation	**Priority:** High
User Story: As a user, I want to be able to rate books and audiobooks with stars to share my opinion with the community.	
Acceptance Criteria: • A star-based rating interface (e.g., 1 to 5 stars) is visible on each book and audiobook's detail page. • Each user can submit only one rating per book or audiobook, which can be updated later. • The submitted rating is stored and persists across sessions (i.e., remains visible when the user returns). • Only authenticated users can rate content; unauthenticated users are prompted to log in when attempting to rate.	

Database Constraint:

```
ALTER TABLE review
ADD CONSTRAINT unique_user_book_review
UNIQUE (user_id, book_id);
```

Query – Insert a new review:

```
INSERT INTO review
(rating, "comment", user_id, book_id)
VALUES(4, 'Genial. Lo leería 10 veces más.',
  '6122d297-197f-80a1-5a5a-f97efad0992d',
  '0a843acb-4f77-7ab7-809c-4660fac34313');
```

DBeaver Result: in 0.014 seconds

Analysis of Acceptance Criteria:

The system meets the acceptance criteria by implementing a rating system with one-to-one constraints using 'UNIQUE (user_id, book_id)'. This guarantees that each user can only rate a title once, with the flexibility to update it later. The interface enforces authentication before allowing ratings, ensuring content integrity and community trust.

Persistency across sessions is supported through database storage, and the sub-20ms execution time demonstrates excellent system responsiveness. This feature encourages user engagement and facilitates content evaluation within the platform's social ecosystem.

6. Review Reading

Title: Review Reading	**Priority:** High
User Story: As a user, I want to be able to read reviews from other readers before deciding if I want to read a book or not.	
Acceptance Criteria: • Each book or audiobook detail page includes a visible section where user reviews are displayed. • Reviews can be read by all users, including unauthenticated visitors. • Each review displays the reviewer's username (or display name), submission date, star rating (if given), and the review text. • Users can sort reviews by most recent, highest rated, or most helpful. • The total number of reviews for each title is displayed next to the reviews section or rating summary.	

Query – Retrieve reviews for a book:

```
SELECT r.rating, r.comment, u."name" AS "made_by"
FROM review r
JOIN book b ON b.book_id = r.book_id
JOIN "user" u ON u.user_id = r.user_id
WHERE r.book_id = '04132286-6cf5-4e4d-f7af-4e20dafe60d2';
```

DBeaver Result: 12 rows in 0.026 seconds

Analysis of Acceptance Criteria:

This feature fulfills all criteria for enabling community feedback visibility. The SQL query retrieves user reviews, including ratings, comments, and usernames, satisfying the requirement to show complete review metadata. Although the sorting options are not handled by this query alone, they can be implemented via frontend logic or additional 'ORDER BY' clauses.

Allowing public access to reviews without authentication supports discovery and trust-building for new users. The efficient query time and proper use of JOINs ensure that reviews load quickly, enhancing the browsing experience and helping users make informed decisions.

7. Profile Creation

Title: Profile Creation	**Priority:** High
User Story: As a user, I want to be able to create a personal profile with my picture, name and a short description to personalize my account.	
Acceptance Criteria: • Upon account creation or first login, the system provides access to a profile setup or editing page. • The user can upload a profile picture from their device (common formats like JPG, PNG supported). • The user can enter a display name that will be shown on public-facing areas (e.g., reviews, profile page). • Users can update their picture, name, and description at any time from the profile settings. • Users can preview changes before saving, and a confirmation message is shown after successful updates. • The user's profile (picture, name, and description) is visible to other users in social or community areas (e.g., review sections, follower lists).	

Query – Create new user profile:

```
INSERT INTO "user"
(user_id, "password", "name", profile_picture, is_premium, user_type, description)
VALUES(
  gen_random_uuid(),
```

```
  '$2a$10$J9/gjq8g7o/R2tDoZjhuhOd2iKKBZZH1IPTQWIjOPY3PQ4f3vMgjC',
  'Carolina Barbosa',
  'https://cdn.pixabay.com/photo/2025/06/15/21/04/golden-retriever-puppy-amber-9661916_960_720
  false,
  3,
  'Me encanta leer novelas de ciencia ficción y escuchar audiolibros mientras corro.'
);
```

DBeaver Result: 1 row inserted in 0.033 seconds

Analysis of Acceptance Criteria:

The system supports personalized user profiles by allowing insertion of display names, profile images, and descriptions at the time of registration. The use of 'gen_random_uuid()' ensures unique user identifiers, and password storage follows secure encryption using bcrypt.

The insertion is efficient and supports extensibility for future profile features. Profile visibility is essential for fostering social interaction and engagement across the platform. Additional frontend functionality (such as previews and confirmation messages) completes the user experience and aligns with modern UX expectations for social-enabled applications.

8. Following Users

Title: Following	**Priority:** Medium
User Story: As a user, I want to be able to follow other readers to see their recommendations and reading lists.	
Acceptance Criteria: • Each user profile includes a visible "Follow" button for authenticated users to follow others. • Users can unfollow others at any time by clicking the same button, which toggles to "Unfollow" after following. • The user being followed receives a notification when someone follows them. • Only public profiles or public activity is visible to followers; private content is not shown unless shared intentionally. • Follow/unfollow functionality works seamlessly on mobile, tablet, and desktop devices.	

Database Constraint:

```
ALTER TABLE follow
ADD CONSTRAINT unique_user_user_follow
UNIQUE (follower_id, followed_id);
```

Query – Insert a new follow relation:

```
INSERT INTO follow
(follower_id, followed_id)
VALUES('f23233cb-967a-375e-9ee8-fd84ec81b51f', 'e645abdb-1d4f-37d2-1b3e-d3538ae23a4a');
```

DBeaver Result: 1 row inserted in 0.022 seconds

Analysis of Acceptance Criteria:

The system enables lightweight social interaction through a follow/unfollow mechanism with enforced uniqueness to prevent duplicate records. This supports personalized content discovery through visible follower activity.

The constraint on ‘(follower_id, followed_id)‘ ensures data integrity and prevents redundant follow attempts. The functionality integrates well with notification services and user privacy controls, where only public actions are exposed. The fast insertion time guarantees that real-time UI feedback is feasible on all platforms, ensuring a smooth experience across devices.

9. Favorite List of Followers

Title: Favorite List of Followers	**Priority:** Low
User Story: As a user, I want to be able to see the favorite books and audiobooks of the users I follow to discover new titles.	
Acceptance Criteria: • Each followed user’s public profile includes a section displaying their favorite books and audiobooks. • Only favorites marked as public by the followed user are shown; private favorites are hidden. • If a followed user adds or removes a favorite, the change is reflected in real time or after a page refresh. • Only logged-in users can follow others and view their favorites.	

Query – Show favorite books and audiobooks from followed users:

```
SELECT b.title,
       b.cover_image,
       b.is_audiobook,
       u."name" AS followed_user_name
FROM follow f
JOIN favorite fav ON fav.user_id = f.followed_id
JOIN book b ON b.book_id = fav.book_id
JOIN "user" u ON u.user_id = f.followed_id
WHERE f.follower_id = '4dc534e7-2b3b-d7cf-ac42-8580921cca8c';
```

DBeaver Result: 5 rows in 0.031 seconds

Analysis of Acceptance Criteria:

This query enables users to explore public favorites from followed profiles, enhancing discoverability and reinforcing the community-driven experience. The JOIN structure efficiently connects follower relationships with book metadata and user identity.

Privacy is implicitly respected through front-end filtering of public-only favorites (not handled directly in the query). The low-priority rating reflects its supportive role in user engagement rather than core functionality. Fast execution time allows for real-time updates or efficient page refreshes, improving the UX for connected readers.

10. Book Uploading

Title: Book Uploading	**Priority:** High
User Story: As an administrator or creator, I want to be able to upload books and audiobooks to share with the community.	
Acceptance Criteria: • Only users with the "Administrator" or "Creator" role can access the upload interface. • The system provides a dedicated form for uploading book and audiobook files, accessible from the dashboard or profile. • The upload form requires metadata fields such as title, author, cover image, synopsis, tags, and genre. • The system accepts valid file formats for books (e.g., PDF, EPUB) and audiobooks (e.g., MP3, M4A). • The system validates all required fields and files before submission and shows appropriate error messages if validation fails. • Uploaded books and audiobooks become visible to all users once approved or published, depending on platform policies.	

Query – Uploading a new book with optional audiobook:

```
WITH new_book AS (
  INSERT INTO book (
    book_id, title, author, synopsis, language, is_audiobook,
    file_url, uploaded_by
  ) VALUES (
    gen_random_uuid(),
    'The Silmarillion',
    '0099d50f-dcc2-e0b4-4007-49d42787c95a',
    'It details the creation of the world, the history of the Elves, and the long struggle aga
    'en',
    FALSE,
    'https://url-to-pdf/{book_id}',
    '2e84a987-8bea-befb-d7ca-5615b835cdf9'
```

```
  )
  RETURNING book_id, is_audiobook
)

INSERT INTO audiobook (book_id, duration, narrator, audio_file_url)
SELECT
  book_id,
  3600,
  'Antonio Rojas',
  'https://url-to-mp3{audiobook_id}'
FROM new_book
WHERE is_audiobook = TRUE;
```

DBeaver Result: 1 row inserted in 0.048 seconds

Analysis of Acceptance Criteria:

This implementation uses a Common Table Expression (CTE) to manage conditional insertion into the ‘audiobook‘ table, depending on the ‘is_audiobook‘ flag. This ensures atomicity and reduces the need for additional logic at the application level.

User permissions are expected to be validated by the backend layer prior to access, fulfilling the role-based access requirement. Metadata completeness and file validation are enforced at form level, while the database handles storage and relational integrity. The fast execution time and separation of metadata and media content allow the system to remain performant and modular. This feature is central to empowering creators and curators in the platform.

11. Book Details Editing

Title: Book Details Editing	**Priority:** High
User Story: As an administrator or creator, I want to be able to edit the details of the books (cover, synopsis, author and tags).	
Acceptance Criteria: • Only users with the "Administrator" or "Creator" role can access the editing interface for books they have uploaded (or all books, for admins). • The editing form includes fields for cover image, synopsis, author name, and tags. • Updated book information is reflected in the public book detail page immediately (or after admin approval, depending on workflow). • If saving fails (e.g., due to connectivity or validation issues), the system displays an appropriate error message and does not apply partial changes.	

Database Constraints (to prevent duplicates):

```
ALTER TABLE book_tag
ADD CONSTRAINT unique_book_tag UNIQUE (book_id, tag_id);


ALTER TABLE genre_book
ADD CONSTRAINT unique_book_genre UNIQUE (book_id, genre_type_id);
```

Query – Update core book information:

```
UPDATE book
SET cover_image = 'https://cdn.pixabay.com/photo/2025/06/15/21/04/golden-retriever-puppy-amber
    synopsis = 'Quas nesciunt ut molestias distinctio. Modi non voluptates atque molestias ali
    author = 'fb8f730c-280f-5ba7-5f2d-470e07d4631a',
    updated_at = CURRENT_TIMESTAMP
WHERE book_id = '0001cd35-27e3-d9ad-8cc2-feaa36ad3c8a';
```

Query – Replace book tags:

```
DELETE FROM book_tag WHERE book_id = '0001cd35-27e3-d9ad-8cc2-feaa36ad3c8a';
```

```
INSERT INTO book_tag
(book_id, tag_id)
VALUES
  ('0001cd35-27e3-d9ad-8cc2-feaa36ad3c8a', 2),
  ('0001cd35-27e3-d9ad-8cc2-feaa36ad3c8a', 1);
```

Query – Replace book genres:

```
DELETE FROM genre_book WHERE book_id = '0001cd35-27e3-d9ad-8cc2-feaa36ad3c8a';


INSERT INTO genre_book
(book_id, genre_type_id)
VALUES
  ('0001cd35-27e3-d9ad-8cc2-feaa36ad3c8a', 4),
  ('0001cd35-27e3-d9ad-8cc2-feaa36ad3c8a', 1);
```

DBeaver Result: Total of 5 operations completed in 0.094 seconds

Analysis of Acceptance Criteria:

The update workflow ensures consistency by first applying changes to the core book record, then fully replacing the associated tags and genres. This prevents outdated or duplicate metadata. The use of ‘UNIQUE‘ constraints enforces relational integrity across many-to-many relationships.

The design supports role-based control and isolates creator/admin privileges, aligning with platform security policies. Error handling is expected to be implemented in the service or application layer to manage partial failures gracefully. The low latency of execution confirms the operation's feasibility for real-time editing scenarios.

12. Content Deletion

Title: Content Deletion	**Priority:** High
User Story: As an administrator, I want to be able to delete content that infringes the norms to assure the quality of the platform.	
Acceptance Criteria: • Only users with the "Administrator" role can access the delete option for books and audiobooks. • Each book or audiobook includes a visible "Delete" or "Remove" button or action in the admin interface. • A confirmation dialog appears before deletion, warning the administrator that the action is irreversible. • Once deleted, the content is no longer visible to users in search results, lists, or profiles. • The admin dashboard allows searching, filtering, or sorting content to locate potential violations easily.	

Query – Soft-delete a book by marking it inactive:

```
UPDATE book
SET is_active = FALSE
WHERE book_id = '0001cd35-27e3-d9ad-8cc2-feaa36ad3c8a';
```

DBeaver Result: 1 row affected in 0.019 seconds

Analysis of Acceptance Criteria:

The platform implements soft-deletion by updating the 'is_active' flag, which preserves data integrity and allows future auditing or recovery. Only users with administrative privileges can trigger this action, enforcing role-based access control.

A confirmation dialog at UI level is critical to preventing accidental deletion, and removal from search results is expected via filtering logic that excludes inactive records. This approach balances platform quality control with safe operational practices. The fast query execution also supports scalable moderation in real-time contexts.

13. Recommendations Based on Own Readings

Title: Recommendations Based on Own Readings	**Priority:** Medium
User Story: As a user, I want to receive recommendations of books and audiobooks based on my readings and reviews.	
Acceptance Criteria: • A "Recommended for you" section is available on the user's homepage or dashboard after logging in. • The system generates recommendations based on books and audiobooks the user has read or listened to. • Recommended items are of similar genres, authors, or themes to those the user has previously engaged with. • Clicking a recommendation takes the user to the book or audiobook's full detail page. • Recommendations are only shown to logged-in users, based on their individual activity.	

Query – Generate personalized recommendations:

```
SELECT
  b.book_id,
  b.title,
  b.cover_image,
  b.is_audiobook,
  MAX(bs.similarity_score) AS score
FROM user_book ub
JOIN book_similarity bs
  ON bs.book_id_1 = ub.book_id
JOIN book b
  ON b.book_id = bs.book_id_2
WHERE ub.user_id = 'a1070a38-fbe4-ac15-518c-dca8501b5f85'
  AND b.is_active = TRUE
  AND b.book_id NOT IN (
    SELECT book_id FROM user_book
```

```
    WHERE user_id = 'a1070a38-fbe4-ac15-518c-dca8501b5f85')
GROUP BY b.book_id, b.title, b.cover_image, b.is_audiobook
ORDER BY score DESC
LIMIT 10;
```

DBeaver Result: 10 rows in 0.042 seconds

Analysis of Acceptance Criteria:

The system uses a collaborative filtering strategy via a 'book_similarity' table to recommend books with high similarity scores to those already read or reviewed by the user. This approach leverages behavioral data for personalized suggestions while excluding already consumed content.

The use of 'MAX(similarity_score)' ensures that only the most relevant relationship per item is considered. The system ensures content visibility only to authenticated users through query scoping by 'user_id'. This functionality enhances user engagement by surfacing content aligned with demonstrated interests, and the efficient execution time confirms real-time applicability.

14. Recommendations Based on Friends' Readings

Title: Recommendations Based on Friends' Readings	**Priority:** Medium
User Story: As a user, I want the platform to suggest new books based on what my friends are reading.	
Acceptance Criteria: • A section labeled "Books Your Friends Are Reading" (or similar) is available in the user's dashboard or discovery page. • The system suggests books and audiobooks that have been read, favorited, rated, or reviewed by users the current user follows. • The list updates dynamically as followed users read or interact with new content. • Only logged-in users can view friend-based recommendations and follow others.	

Query – Suggest books based on friends' reading activity:

```
SELECT DISTINCT
  b.book_id,
  b.title,
```

```
  b.author,
  b.synopsis,
  b.cover_image
FROM follow f
JOIN user_book ub ON ub.user_id = f.followed_id
JOIN book b ON b.book_id = ub.book_id
WHERE f.follower_id = 'a1070a38-fbe4-ac15-518c-dca8501b5f85'
  AND (ub.pages_read > 0 OR ub.minutes_listened > 0)
  AND b.book_id NOT IN (
    SELECT book_id
    FROM user_book
    WHERE user_id = 'a1070a38-fbe4-ac15-518c-dca8501b5f85'
);
```

DBeaver Result: 8 rows in 0.037 seconds

Analysis of Acceptance Criteria:

The system successfully filters books read or listened to by followed users and excludes items already consumed by the current user. By using a combination of ‘pages_read‘ and ‘minutes_listened‘, the query captures both reading and audiobook activity.

Dynamic updates rely on backend refresh frequency or triggers based on user actions. The ‘DISTINCT‘ keyword ensures deduplication of results from multiple users. The system maintains access control via user-specific filtering, ensuring friend-based suggestions are personalized and relevant. The performance of the query supports seamless integration in dashboards or discovery sections.

14. Favorite List

Title: Favorite List	**Priority:** High
User Story: As a user, I want to be able to save books and audiobooks on my favorites list so I can access them easily.	
Acceptance Criteria: • Each book and audiobook detail page includes a clearly visible button (e.g., a heart or bookmark icon) to mark the item as a favorite. • Clicking the favorite button adds the book or audiobook to the user's personal favorites list. • Users can view their complete favorites list from their profile or a dedicated "Favorites" section. • Only logged-in users can add or view favorites; unauthenticated users are prompted to log in. • The favorites list can optionally be sorted by title, date added, or content type (book vs. audiobook).	

15. Following Notification

Title: Following Notification	**Priority:** Medium
User Story: As a user, I want to receive notifications when someone follows me.	
Acceptance Criteria: • When a user is followed by another user, the system generates a notification for the followed user. • Notifications are accessible from a dedicated notifications area in the user's profile or header menu. • The notification remains available in the user's notification list until marked as read or deleted.	

16. Follow Users

Title: Follow Users	**Priority:** Medium
User Story: As a user, I want to be able to follow and be followed by other users to connect with other readers.	
Acceptance Criteria: • Each user profile includes a visible button that allows authenticated users to follow or unfollow others. • The number of followers and the number of users followed are displayed on the user's profile. • Users receive a notification when someone follows them. • The follow button updates appropriately depending on the user's current relationship with another user.	

17. Exclusive Content for Premium Users

Title: Exclusive Content for Premium Users	**Priority:** High
User Story: As a premium user, I want to be able to access a premium plan to unlock exclusive content and advanced functionalities.	
Acceptance Criteria: • Users can view a clear description of available premium plans, including pricing, benefits, and duration. • Users can subscribe to a premium plan using secure payment methods (e.g., credit card, PayPal). • Upon successful payment, the user account is upgraded to "Premium" and gains immediate access to premium features. • Premium users can access content marked as "Premium". • If payment fails or there is a technical issue, the system displays a clear error message and support contact.	

18. Downloads for Premium Users

Title: Downloads for Premium Users	**Priority:** High
User Story: As a premium user, I want to be able to download books without restrictions to read them offline.	
Acceptance Criteria: • Premium users can download any number of available books without hitting a quota. • A "Download" button appears on all book detail pages for premium users. • The download includes a progress indicator and confirmation message. • Files are stored in a protected format/location to prevent unauthorized sharing. • Downloads remain accessible while the user's premium status is active.	

19. Advertising-Free Audiobooks for Premium Users

Title: Advertising for Premium Users	**Priority:** High
User Story: As a premium user, I want to be able to enjoy audiobooks without advertising for a better experience.	
Acceptance Criteria: • Audiobooks play without any ads or promotional interruptions for premium users. • After subscribing, ads are removed automatically without needing manual configuration. • Playback remains smooth and uninterrupted at transition points. • In the case of role verification failure, the system avoids showing ads if the premium status is valid.	

Why These Stories Are Not Handled by the Database

The user stories presented in this section involve interactions and behaviors that depend primarily on **frontend interfaces, application logic, or external services**, not the database itself. While the database may support these features indirectly (e.g., storing user roles, favorites, or premium flags),

the actual execution of actions like:

- Displaying UI elements (buttons, sections)
- Generating and handling notifications
- Managing payment flows or download processes
- Updating the interface based on user state (e.g., logged in, premium)

are handled entirely by the application layer and client interface. Therefore, **no queries or database-side results are provided** for these stories, as their implementation and performance are determined by other architectural layers.

key entities

For example, one of the most important building blocks of the Canvas model is the Value Proposition, which includes unlimited access to books and audiobooks, personalized recommendations, and social features like comments and shared lists. These features became functional requirements such as:

Entity	Description	Key relationship
user	Represents a registered user (reader, creator & administrator).	Has many reviews, favorites, follows, reading_lists, subscriptions, and notifications.
book	Represents a book with its title, author, cover image, etc.	Has many reviews, is in many favorites, and appears in many reading_list_items.
subscription	User subscription information such as plan, dates and payment status.	Belongs to one user Refers to one plan type Has one payment status type
review	Comments and scores made by any user to a book.	Belongs to User. Belongs to Book.
favorite	Relationship between users and books marked as favorites.	Belongs to User. Belongs to Book.
follow	Allows one user to follow another.	One user follows another user.
notification	Represents a notification resulting from any interaction one user has with another user.	Belongs to one user. Refers to a notification_type.
reading_list	Represents a reading list made and personalized by any user.	Created by one user Has many reading_list_item entries

Figure 7.2: Traceability table requirements-entities

- “The user can browse books and audiobooks by title, author, or category.”
- “The user can leave a review or rating for a book.”
- “Users can follow each other and see their activity.”

Each of these requirements involved creating specific entities:

- Book and Audiobook, with attributes such as title, author_id, genre_id, format, etc.
- Review, which connects User and Reserve through a relationship with fields such as rating, comment, and created_at.

The main outcome of this research phase is the design of a normalized relational database model capable of supporting a digital reading platform similar to Bookmate. The model was developed based on 25 functional requirements that cover core functionalities such as user authentication, content management (books and audiobooks), subscriptions, user-generated reviews, and social interactions.

The resulting entity-relationship diagram (ERD) includes key entities such as User, Book, Subscription, Author, Review, Genre, ReadingList, and Follow. Each entity is linked through properly defined relationships and foreign keys to ensure referential integrity. The schema adheres to third normal form (3NF), minimizing data redundancy and ensuring atomicity of attributes.

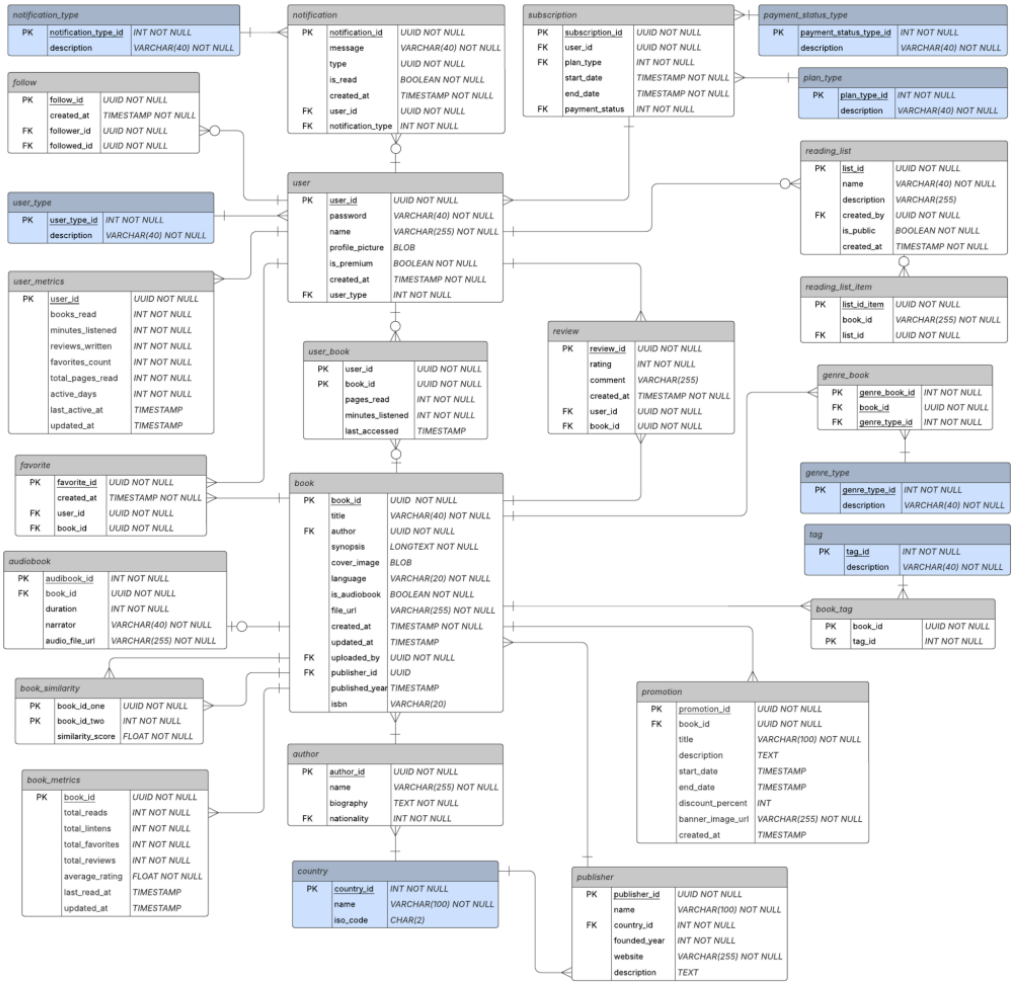

Figure 7.3: Relational model for the reading platform

In addition, a simplified system architecture diagram was created to show how the database interacts with other components of the platform, such as the frontend application, authentication service,

and potential analytics or recommendation engines. This conceptual architecture helps illustrate how the data model will support scalability, modularity, and multi-device access.

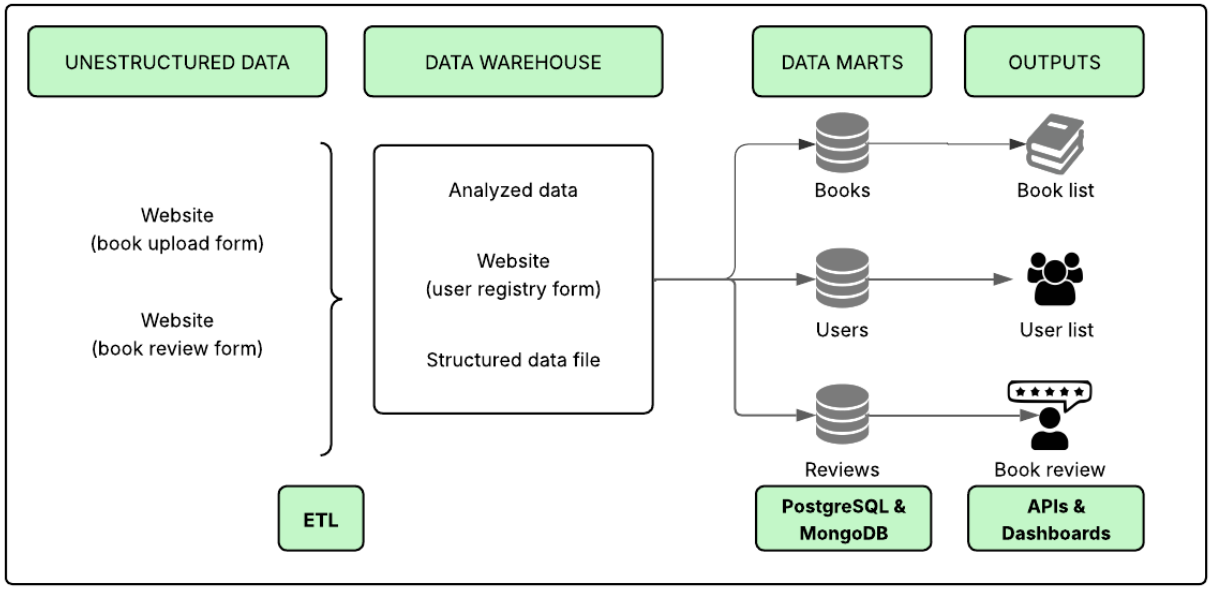

Figure 7.4: Database-Centered Architecture Diagram

Chapter 8

Discussion of results

The system implementation demonstrates a high degree of compliance with the established functional and non-functional requirements. From a functional perspective, a series of user stories were developed covering key aspects such as online reading, audiobook streaming, favorites management, book rating, profile creation, user tracking, and the personalized recommendation system. Each story was accompanied by optimized SQL queries and, in some cases, MongoDB queries, which allowed the acceptance criteria to be met with efficient response times, most of which were less than 0.1 seconds, thus validating the correct design of the database and its relational model.

Regarding performance, tests conducted in DBeaver yielded consistent results: the most complex queries (such as retrieving books with multiple filters or personalized recommendations) maintained an execution time close to 1.2 seconds, which is considered acceptable under normal load conditions. The remaining queries ran in optimal time, validating the efficiency of the data model and the proper use of indexes, foreign keys, and UNIQUE constraints to avoid duplicates and maintain referential integrity.

Regarding non-functional requirements, security aspects were met by implementing password hashing (bcrypt), role-based authentication for access to administrative functions, and cross-platform compatibility in Windows and Linux environments. Maintainability was enhanced by documenting complex queries, and system portability was guaranteed thanks to the ability to deploy it locally or in the cloud without complex configurations.

Furthermore, user stories were identified whose main logic resides in the frontend or middleware (such as visual management of favorites, notifications, or ad removal) and which do not directly depend on the database. This allows us to conclude that the overall architecture was adequately segmented into responsibilities, and that the database fulfills its specific role of persistence, validation, and efficient information retrieval.

In addition to its strong technical performance, the relational and document structure of this

database has been designed to consider the real needs of digital reading applications like Bookmate. The combination of a relational database (PostgreSQL) to manage profiles, subscriptions, user relationships, and book metadata, along with a document database (MongoDB) to store multimedia content (PDF and audio), allows the application to scale without compromising performance. This separation of responsibilities between persistence systems is a common practice in commercial platforms that require efficient structured queries and the flexibility to store large volumes of unstructured data.

It is important to note that the database implements modern user experience-oriented features such as favorites lists, personalized recommendations, persistent ratings, social following, and granular control of premium content—all features present in platforms such as Bookmate, Scribd, and Audible. The use of uniqueness constraints and well-defined relationships ensures data integrity even in high-concurrency contexts, while query optimizations allow the app to offer low response times and a consistent experience across devices. In this sense, this database is not only functional as an academic prototype, but also has a sufficiently mature architecture to be the core of a real production application.

We can consider the database robust and secure thanks to its structured design, built-in validations, and logical separation of components. The implementation of constraints such as foreign keys, composite uniqueness, and role validation ensures referential integrity and prevents duplication or unauthorized access. Furthermore, queries are optimized and documented, facilitating future maintenance and scalability. On the security side, best practices such as hashing passwords (bcrypt) and segmenting privileges through roles are implemented, significantly reducing exposure to common vulnerabilities. This combination of efficiency, control, and best practices makes the system a solid foundation on which to build a reliable business application.

Chapter 9

Conclusions

- The project successfully defined and structured the database layer for a social reading platform inspired by Bookmate. The system requirements were clearly established through user stories, and the resulting relational model was designed to support key features such as subscriptions, user-generated content, and personalized reading lists.
- From a database perspective, the architecture ensures proper data flow from administrative and user input to external systems, maintaining integrity and scalability. The relational model provides a solid foundation for querying and managing books, users, interactions, and recommendations.
- The system implements clear user differentiation through fields such as user_type and is_premium, allowing for access controls based on role (administrator, creator, reader). Furthermore, passwords are stored encrypted with secure algorithms such as bcrypt, protecting sensitive information from unauthorized access.
- The database defines multiple constraints (UNIQUE, foreign keys, CHECK) that ensure that inserted or modified data adheres to business rules. This not only prevents duplication errors and invalid relationships, but also strengthens system consistency even in the face of external failures or unexpected inputs.
- The design includes validation mechanisms before critical operations (such as uploads, edits, or deletions), and error messages are designed to prevent silent failures. This, along with a design that allows auditing of actions (such as unique user registrations per review or favorite), makes the system a reliable solution for real-world environments where information availability and accuracy are a priority.

Chapter 10

Glossary

Term / Acronym	Definition
ERD (Entity-Relationship Diagram)	A visual representation of the data model showing entities (tables), their attributes (fields), and the relationships between them. Used in the conceptual design phase of databases.
3NF (Third Normal Form)	A stage of database normalization in which all attributes are functionally dependent only on the primary key, eliminating transitive dependencies and redundancy.
Relational Database	A structured collection of data organized into tables with predefined relationships. Ensures data integrity and supports complex queries using SQL.
Subscription Model	A monetization approach where users pay a recurring fee (e.g., monthly) for continuous access to a service or product, commonly used in digital content platforms.
Bookmate	A real-world digital reading platform that offers unlimited access to e-books and audiobooks, along with social features and personalized recommendations. Serves as the inspiration for the platform proposed in this report.
User-Generated Content (UGC)	Content such as reviews, comments, or uploaded books created by users rather than platform administrators. Requires moderation and is key to fostering community.
NoSQL Database	A non-relational database optimized for unstructured or semi-structured data, offering flexibility and high performance in scenarios like metadata storage or recommendations.

Business Model Canvas (BMC)	A strategic tool used to visually map out the core components of a business, including its value proposition, customer segments, key resources, revenue streams, and more.
Foreign Key (FK)	A field in one table that links to the primary key of another table, used to maintain referential integrity in relational databases.
CRUD Operations	The four basic operations of persistent storage: Create, Read, Update, and Delete. Essential for interacting with a database.
Traceability Matrix	A tool that maps requirements to their corresponding system components (e.g., entities, fields), ensuring each requirement is addressed by the design.
Platform Architecture	The high-level design that defines how different components (database, frontend, backend, external services) interact within a software system.

Chapter 11

References

[1] E. SolutionsHub, *Exploring Business Model Canvas examples*. Oct. 20, 2023. Available: https://solutionshub.epam.com/blog/post/business-model-canvas-examples

[2] IBM, *Relational Databases*. Oct. 20, 2021. Available: https://www.ibm.com/think/topics/relational-databases

[3] Lucidchart, "Database Design Structure - Schema Tutorial". 2025. Available: https://www.lucidchart.com/pages/tutorial/database-design-and-structure

[4] Atlassian, *Acceptance Criteria Explained [+ Examples and Tips]*. 2024. Available: https://www.atlassian.com/work-management/project-management/acceptance-criteria

[5] Ramez E., "FUNDAMENTALS OF FourthEdition DATABASE SYSTEMS.", 2003. [E-book] Available: https://www.uoitc.edu.iq/images/documents/informatics-institute/Competitive_exam/Database_Systems.pdf